

Sistema de Agendamentos da Clínica Vida Plena

Contexto

A Clínica Vida Plena tem 6 médicos e 2 recepcionistas. Cerca de 28% das consultas agendadas terminam em falta, e isso está afetando o faturamento.

Seu desafio é construir um **sistema de agendamento de consultas** para a clínica e, a partir dos dados históricos, uma funcionalidade que ajude a reduzir as faltas.

Conversamos com a equipe:

Dra. Marta (dona)

"Paciente falta e não avisa, o horário fica vazio. Quero cobrar multa de quem falta. Também pensei em overbooking, igual companhia aérea. Nossa agenda vive cheia, o pessoal espera umas três semanas por um horário."

Juliana (recepcionista)

"Ligo para todo mundo no dia anterior confirmando, mas metade não atende. Tem gente que confirma e falta do mesmo jeito. Outros ligam para cancelar, mas às vezes esqueço de marcar no sistema. Acho que os de convênio faltam mais."

Dr. Paulo (cardiologista)

"Minhas segundas de manhã são as piores. E paciente novo falta muito mais que paciente antigo."

Arquivos de entrada

- `agendamentos.csv`: consultas dos últimos 12 meses e as já marcadas para as próximas semanas, exportadas do sistema antigo, preenchido à mão, na data de envio deste desafio. **Não assumo que os dados estão limpos.**
- `medicos.json`: médicos, especialidades e grade de horários.

Colunas do CSV:

Coluna	Descrição
<code>id</code>	Identificador do agendamento
<code>paciente_id</code>	Identificador do paciente
<code>paciente_nome</code>	Nome do paciente
<code>paciente_telefone</code>	Telefone do paciente
<code>tipo_atendimento</code>	Convênio ou particular
<code>medico_id</code>	Identificador do médico
<code>data_agendamento</code>	Quando a consulta foi marcada

Coluna	Descrição
data_consulta	Data e hora da consulta
status	Status registrado no sistema antigo

Datas no fuso America/Sao_Paulo.

Stack

Node.js com TypeScript no backend, MongoDB e React com TypeScript no frontend. Bibliotecas à sua escolha.

Parte 1: Sistema de agendamentos

Regras

- As consultas ocupam slots de 30 minutos dentro da grade do médico.
- Um médico não pode ter duas consultas no mesmo slot, e um paciente não pode ter duas consultas no mesmo horário.
- realizada e falta só podem ser registradas depois do horário da consulta. Cancelamentos só podem ser registrados antes dele.
- Status finais (realizada, falta, cancelada_paciente, cancelada_clinica) não mudam mais.
- Transições inválidas devem ser rejeitadas com erro claro.

Status e transições permitidas:

Status atual	Pode ir para
agendada	confirmada, realizada, falta, cancelada_paciente, cancelada_clinica
confirmada	realizada, falta, cancelada_paciente, cancelada_clinica
realizada, falta, cancelada_paciente, cancelada_clinica	Nenhum (status final)

O que construir

- 1 **Importação:** um comando que carrega os dois arquivos, trata os problemas dos dados e gera um relatório do que foi importado, corrigido ou descartado (e por quê).
- 2 **Agendamento:** criar consultas e alterar seus status seguindo as regras acima.
- 3 **Indicadores de falta,** filtráveis por período: a taxa geral, a taxa por médico e os recortes que, na sua análise, ajudam a explicar as faltas.
- 4 **Frontend:** uma tela de indicadores, com filtro por período.

Médicos e pacientes vêm dos arquivos; não é preciso cadastrá-los. Também não é preciso autenticação.

Decisões que você precisa tomar

As regras abaixo não foram definidas pela clínica. Escolha, implemente e justifique no README:

- 1 Se o paciente cancela em cima da hora, isso conta como falta? A partir de quantas horas antes?
- 2 Consultas canceladas entram no cálculo da taxa de falta? Exemplo: 10 consultas, 2 faltas e 2 cancelamentos resultam em 20% (2 de 10) ou 25% (2 de 8)?
- 3 "Primeira consulta" significa a primeira vez do paciente na clínica ou a primeira com aquele médico?
- 4 Quando o CSV tem registros duplicados ou conflitantes, qual deles vale?
- 5 O sistema deve fazer algo com pacientes que faltam com frequência? O quê?

Parte 2: Funcionalidade para reduzir faltas

Analise os dados, descubra o que mais explica as faltas e construa uma funcionalidade que ataque isso.

- A funcionalidade deve funcionar de ponta a ponta (API e tela), como algo que a recepção ou o paciente usaria no dia a dia. Um relatório ou análise sozinho não conta.
- Envios de WhatsApp, SMS ou e-mail devem ser simulados, sem integração real.
- Prefira fazer uma coisa bem feita a várias pela metade.

Qualidade

Trate o código como algo que iria para produção. Avaliamos, entre outras coisas, testes automatizados, tratamento de erros, organização do código e a qualidade visual e de usabilidade do frontend.

O projeto completo (backend, frontend e MongoDB) deve subir com um único comando, como `docker compose up`.

Você pode usar IA livremente.

Entrega

Um repositório Git com o código e um `README.md` contendo:

- como rodar o projeto, importar os dados e executar os testes;
- o que os dados mostraram;
- suas decisões para os 5 pontos da Parte 1 e o porquê;
- o que você construiu na Parte 2, por que escolheu isso e qual resultado espera;
- quantas faltas por mês a sua proposta deve evitar, e a conta que levou a esse número;
- como a clínica saberia, em 3 meses, se funcionou;
- o que ficou de fora e os riscos da sua proposta;
- como você usou IA e onde precisou corrigir o que ela gerou.

Se algo não estiver claro, tome uma decisão e registre-a no README.